University of Kansas

Algorithmic Algebraic Parser
Software Requirements Specifications

Version <1.6>

Revision History

Date	Version	Description	Author
<dd/mm/yy>	<x.x>	<details>	<name>
2/24/26	1.0	Imported .docx template and established base document structure.	Shlok Prabhu
2/24/26	1.0.1	Drafted initial Use-Case models and identified actors.	Shlok Prabhu
3/2/26	1.1	Finished parts of section 1.	Spencer Weishaar
3/3/26	1.2	Completed Overall Description and defined software/hardware constraints.	Lincoln Earnshaw
3/3/26	1.3	Outlined core Functional Requirements (3.1.1 - 3.1.4).	Nathan Phillips
3/3/26	1.3.1	Added requirements for unary operations, parenthesis handling, and numeric constants.	Simeon Geib
3/3/26	1.4	Drafted Error Handling (3.1.8) and Supplementary Requirements (3.3).	Zema Samuel
3/3/26	1.4.1	Updated Use Cases to reflect new error handling scenarios.	Shlok Prabhu
3/10/26	1.5	Reviewed all sections, formatted tables, and finalized sections 1, 3 for baseline.	Shlok Prabhu, Nathan Phillips
3/10/26	1.6	Finalized content in sections 2, 4.	Simeon Geib, Lincoln Earnshaw, Spencer Weishaar, Shlok Prabhu, Zema Samuel

©University of Kansas, 2026

Table of Contents

1. Introduction 4
    1.1 Purpose 4
    1.2 Scope 4
    1.3 Definitions, Acronyms, and Abbreviations 4
    1.4 References 4
    1.5 Overview 4

2. Overall Description 5
    2.1 Product perspective 5
        2.1.1 User Interfaces 5
        2.1.2 Software Interfaces 5
        2.1.3 Memory Constraints 5
    2.2 Product functions 5
    2.3 User characteristics 5
    2.4 Constraints 5
    2.5 Assumptions and dependencies 5

3. Specific Requirements 5
    3.1 Functionality 5
        3.1.1 <Functional Requirement One> 6
    3.2 Use-Case Specifications 6
    3.3 Supplementary Requirements 6

4. Classification of Functional Requirements 6

5. Appendices 6

Software Requirements Specifications

1. Introduction

1.1 Purpose

The SRS describes the behavior of the Algorithmic Algebraic Parser, including the tokenization and the operations of numbers observed in the command line input. The SRS also describes non-functional requirements, design constraints, and other factors in the Algorithmic Algebraic Parser to provide a complete and comprehensive description of the requirements in this project. The SRS will include an overall description of the Algorithmic Algebraic Parser and its functions, including the tokenization and completion of the numbers and operations observed in the command line.

1.2 Scope

This Software Requirements Specifications document applies to the Algorithmic Algebraic Parser, currently in development by our team. The SRS covers all the required functionality needed to complete the Algorithmic Algebraic parser, as well as some desired functionality to be added with ample and sufficient time. The SRS covers Use-Case models pertaining to the functionality of the system, covering and testing all and every aspect of the functionality required for the Algorithmic Algebraic Parser.

1.3 Overview

Other items the SRS contains (not referenced above) include the classification of functionalities, constraints, assumptions, and dependencies, and the overall project description.
The SRS is organized in a typical three-section format, starting with the introduction of the document. The introduction of the document summarizes and introduces the purpose and context of the SRS. Next is the overall project description. This section of the SRS describes aspects of the project, including design, functionality, constraints, and more. The final section covers the specific requirements of the project. This section outlines all functionality requirements the project requires. At the top of the SRS document is the table of contents, and at the bottom is the appendix.

2. Overall Description

2.1 Product perspective

2.1.1 User Interfaces

The primary interface for the parser will be a standard terminal, running from input obtained at the command line.

2.1.2 Software Interfaces

The parser will be modular and have a way to interface easily and seamlessly with the larger compiler project. This project will be easy to interface with for developers.

2.1.3 Memory Constraints

The program should run in a way that doesn't require large amounts of memory. More specifically, it should use less than 1 gigabyte (1 Gb).

2.2 Product functions

The product will be able to:

- Take an input
- Convert the input into appropriate tokens.
- Run correct calculations based on the tokens.
- Output the results
- Gracefully handle any errors.

2.3 User characteristics

The assumed user will primarily be a developer or a piece of software. If they are a developer, they are assumed to have an understanding of programming. This being said, the user does not necessarily need to have these characteristics, as all they will need to do is provide input.

2.4 Constraints

Must be done with the "C++" language. Must run very quickly and with very little memory.

2.5 Assumptions and dependencies

We assume this program will be run on modern hardware that supports C++ and any other necessary dependencies required by it. We assume C++ will be installed in the environment where this is run.

SourceText → Lexer → Tokens → Parser → AST → Evaluator → Output/Logger

3. Specific Requirements

3.1 Functionality

3.1 Functional Requirements

3.1.1 Read Command Line Input

- The system must provide a user-friendly and legible command-line interface.
- The system must allow the user to input a mathematical expression as a string of text.
- The system should support all UTF-8 characters.
- The system should preserve all characters, including white space.
- The system should treat newline characters as the end of input.
- If the command line fails, the system should provide a detailed error message.

User Input	**Expected Result**
2+3\n	System reads the string and passes it to the parsing component.
3 + 4 \n	System preserves white space, reads the string, and passes it to the next component.
Enter pressed with no input	Error is passed to the user.

3.1.2 Parse Expressions and Tokenize

- The system must implement a function to tokenize the input expression.
- The system must create a data structure, such as a stack or a tree, to represent the structure of the parsed expression.
- The system must correctly handle and ignore extraneous whitespace between tokens.

User Input	**Expected Result**

4 * 2	System tokenizes into [4], [*], [2] and builds the representative data structure.

3.1.3 Support Arithmetic Operations

- The system must implement support for the addition (+) operator.
- The system must implement support for the subtraction (-) operator.
- The system must implement support for the multiplication (*) operator.
- The system must implement support for the division (/) operator.
- The system must implement support for the modulo (%) operator.
- The system must implement support for the exponentiation (**) operator.

User Input	**Expected Result**
10 * 2 / 5	System successfully processes the multiplication and division operators. Result: 4.

3.1.4 Enforce Operator Precedence and Associativity

- The system must define and enforce operator precedence according to PEMDAS rules.
- The system must evaluate the operators +, -, *, /, and % using left-to-right associativity.
- The system must evaluate the exponentiation operator ** using right-to-left associativity.

User Input	**Expected Result**
2 ** 3	System calculates 2 raised to the power of 3. Result: 8.
4 * (3 + 2) % 7 - 1	System calculates the result step-by-step strictly following PEMDAS. Result: 5.

3.1.5 Support Unary Operations

- The system must support the unary + operator.
- The system must support the unary - operator.

User Input	Expected Result
-(+1) + (+2)	System correctly applies unary negation and addition operators. Result: 1.

3.1.6 Handle Parentheses

- The system must identify and evaluate expressions enclosed within parentheses.
- The system must use parentheses to determine the correct order of evaluation, overriding default precedence if necessary.
- The system must correctly process both necessary and extraneous nested parentheses without affecting validity.

User Input	Expected Result
8 - (5 - 2)	System evaluates the inner parenthesis first. Result: 5.

3.1.7 Evaluate Numeric Constants

- The system must recognize numeric constants within the input expression.
- The system must initially assume all input values are integers.
- The system must correctly calculate mathematical results using these constants.

User Input	Expected Result
3 + 4	System recognizes 3 and 4 as constants and adds them. Result: 7.

3.1.8 Error Handling and Reporting

- The system must implement robust error handling to manage invalid scenarios.
- The system must provide clear and informative error messages to the user for invalid inputs.
- The system must detect and clearly report division by zero.
- The system must detect and clearly report modulo by zero.
- The system must report unmatched or mismatched parentheses.
- The system must report the use of invalid characters or unsupported operators.
- The system must detect missing operators or missing operands.

User Input	**Expected Result**
4 / 0	System aborts calculation and reports a "Division by zero" error.
2 * (4 + 3 - 1	System reports an "Unmatched parenthesis" error.
7 & 3	System reports an "Invalid character/operator" error.

3.1.9 Output Evaluated Results

- The system must calculate the final result of the parsed arithmetic expression.
- The system must display the calculated results clearly in the command-line interface.

3.1.10 Unit Testing

- The system must include a suite of unit tests.
- The unit tests must verify the correctness of the expression evaluator against expected outcomes.

3.2 Desirable / Optional Requirements

Note: The project guidelines encourage exploring additional features or optimizations beyond the specified baseline requirements to enhance the final product.

- **3.2.1 Floating Point Support:** The system should be built to accommodate future change requests for floating-point input values.
- **3.2.2 Mathematical Functions:** The system may support standard mathematical functions (e.g., sin, cos, ln).

- **3.2.3 Mathematical Constants:** The system may recognize built-in mathematical constants (e.g., PI, e, phi).
- **3.2.4 Scientific Notation:** The system may support parsing and evaluating numbers in scientific notation (e.g., 1E10).
- **3.2.5 Graphical User Interface (GUI):** The system may include a visual interface as an alternative to the command-line interface.
- **3.2.6 History and Persistence:** The system may log previously entered expressions and their calculated results for user reference.
- **3.2.7 AST Visibility (Debug Mode):** The system may include a debug flag that outputs a visual representation of the generated Abstract Syntax Tree (AST) to verify parsing structure.
- **3.2.8 Error Highlighting:** The system may visually indicate the exact location of a syntax error within the input string (e.g., using caret underlines ^).

3.3 Use-Case Specifications

See *SRS-USE-CASE.svg* for modeling.

3.3.1 Use Case: Evaluate Valid Arithmetic Expression

Field	**Description**
Actor	User
Description	The user enters a mathematical expression for the system to evaluate and return a result.
Preconditions	The command-line interface is active and waiting for user input.
Main Success Scenario 1	The user inputs an arithmetic expression consisting of numeric constants and operators.
Main Success Scenario 2	The system tokenizes the input expression.
Main Success Scenario 3	The system creates a data structure, such as a stack or tree, to represent the expression's structure.
Main Success Scenario 4	The system evaluates the expression while considering operator precedence according to PEMDAS rules.
Main Success Scenario 5	The system applies left-to-right associativity for `+`, `-`, `*`, `/`, and `%` operators.
Main Success Scenario 6	The system applies right-to-left associativity for the `**` operator.

Main Success Scenario 7	The system displays the calculated result via the command-line interface.
Alternative Flow: Unary	If the input includes unary + or −, the system correctly processes them.
Alternative Flow: Parentheses	If the input includes parentheses, the system identifies and evaluates those enclosed expressions first to determine the order of evaluation. +1
Postconditions	The correct calculated result is displayed to the user.

3.3 Use Case: Handle Invalid Input and Mathematical Errors

Field	Description
Actor	User
Description	The system intercepts invalid expressions or mathematical errors and safely informs the user without crashing. +1
Preconditions	The command-line interface is active and waiting for user input.
Main Success Scenario 1	The user inputs an invalid expression.
Main Success Scenario 2	The system parses the input and detects an issue, such as unmatched parentheses, division by zero, incorrect operator usage, missing operands, or invalid characters.
Main Success Scenario 3	The system manages the scenario by halting the evaluation process. +1
Main Success Scenario 4	The system reports the error clearly, providing an informative error message to the user. +1

Postconditions	An error message is displayed, and the system recovers, remaining open for a new input string.

3.4 Supplementary Requirements

3.4.1 Development Constraints

- **Implementation Language:** The expression parser and evaluator program must be developed entirely in C++.
- **Programming Paradigm:** The system must be structured using object-oriented programming (OOP) principles.
- **Process Traceability:** The software development process must ensure that each stage aligns directly with the previous one; specifically, the design must reflect the requirements, and the implementation must adhere to the design.

3.4.2 System Quality Attributes (Non-Functional Requirements)

- **Robustness:** The system must safely manage invalid scenarios, such as division by zero or malformed expressions, without crashing, and it must provide clear, informative error messages to the user.
- **Maintainability & Extensibility:** While the system initially assumes integer-only inputs, the architecture must embrace change and be easily extensible to accommodate future requests, such as the inclusion of floating-point values.
- **Readability & Code Quality:** The final product will be evaluated on code quality, meaning the source code must be well-structured and highly readable.

3.4.3 Documentation Requirements

- **Internal Documentation:** The C++ source code must include comprehensive comments and documentation to explain the underlying logic and functionality of the program.
- **Project Deliverables:** The team must produce and submit a detailed project management plan, a requirements document, and a design document.
- **End-User Documentation:** A user manual or a README file containing examples of valid and invalid expressions must be provided to guide the user.

3.4.4 Testing Requirements

- **Unit Testing:** The development process must include a suite of rigorous unit tests specifically designed to verify the correctness of the expression evaluator.
- **Test Plan Deliverable:** A formal test plan must be submitted, which clearly documents the test cases and compares the expected results against the actual results.

4. Classification of Functional Requirements

Functionality	Type
Read Command Line Input	Essential
Parse Expressions and Tokenize	Essential
Support Arithmetic Operations	Essential
Unit Testing	Essential
Error Handling and Reporting	Essential
Output Evaluated Results	Essential
Evaluate Numeric Constants	Essential
Handle Parentheses	Essential
Support Unary Operations	Essential
Enforce Operator Precedence and Associativity	Essential
Floating Point Support	Desirable
Mathematical Functions	Desirable
Mathematical Constants	Desirable
Scientific Notation	Desirable
Graphical User Interface (GUI)	Optional
History and Persistence	Optional
AST Visibility (Debug Mode)	Optional
Error Highlighting	Optional